# Mixed-effects Models

BIOL 3401 Mount Royal University

Topic 9, Winter 2026

## Contents



## Accounting for structure in our models

We have seen that structure is pervasive in biological data. To ensure we can generalize the conclusions from our studies to other places, subjects, or contexts, we often repeat our experiments across multiple units. To rule out variation in experimental conditions that could impact our results, we try to ensure each subject is measured more than once. This is good practice in study design. However, it also creates challenges for us as we analyze our data.

We will discuss these challenges today as we learn how to deal with structure in our statistical models.

## Learning Objectives

After completing this topic, you should feel comfortable:

1. Fitting mixed-effects models using `lme4`
2. Interpreting the summaries of mixed-effects models
3. Assessing the relative importance of between-group and within-group variation
4. Assessing overall goodness of fit for mixed-effects models
5. Visualizing complete pooling, partial pooling, and no pooling across groups

### Set up your R workspace for the day

#### Set your working directory

Make sure we're setting our working directory!

```
setwd()
```

### Install and load packages

We are going to need to load `tidyverse`. We are also going to install and load three new packages:

- `lme4`
- `lmerTest`
- `performance`

Using the code block below, install the new package and load the packages.

### Load data

Today our dataset—**response_times.csv**—contains the results from a reaction time experiment using lab mice, *Mus musculus*. The mice were subjected to a range of ambient temperatures (degrees Celsius) and performed a maze task. The researchers recorded then recorded their force generation delay (in milliseconds) after neural stimulation. Variables include:

- `animal`: The individual identifier for each mouse
- `temp`: The temperature of the cage during the trial (degrees Celsius)
- 'response_time": The force generation delay after neural stimulation (in milliseconds)
- `trials`: The number of trials required to complete the maze task

Load the dataset by finishing the `read_csv()` functions.

```
response_times <- read_csv()
```

## Part 1: Accounting for structure with mixed-effects models

### 1a: Assessing structure

We already know we will have to account for structure because we have multiple mice with multiple observations each. There are 353 observations in our dataset. How many trials were conducted on each mouse?

It's easy to get a sense of individuals x observations using some of the functions we already know from Tidyverse. If you recall, in addition to performing calculations like mean, standard deviation, etc. to summarize variables in tabular data, the `summarize()` function can also count how many observations are in a group when paired with `group_by()` and `n()`.

Here we are treating each mouse as a group, and its trials are individual observations within that group. You could also imagine individual observations as transects within forests, fish within tanks, etc. In each case, there is something linking the observations that makes them non-independent.

```
response_times %>%
    group_by(animal) %>%
    summarize(Count = n()) %>%
    arrange(Count) %>%
    print(n = 25)
```

As you can see, we've grouped our data by individual animal, then used `summarize()` to count (`n()`) the number of observations in each group. We've also used `arrange()` to sort the new *Count* column from smallest to largest. We then asked R to print the summary for all 25 mice.

From the output, we can see that each mouse was observed over anywhere from 10 to 18 trials.

### 1b: Specifying random effects in `lme4`

We discussed two main types of random effects we can specify in mixed-effects models.

**Random intercepts** allow each group to have its own baseline relationship with the explanatory variable. This is equivalent to us saying that each group's value of the response variable is unique when the predictor variable is equal to zero. For example, if we fit a model to test the effect of height on shoe size, we would likely expect pre-schoolers, junior high school students, and MRU students to each have different baseline shoe sizes. We likely wouldn't expect any MRU students to have size 3 shoes!

**Random slopes** allow each group's relationship between the predictor and response variable to vary. These work a bit like interaction terms. As an example, we might expect a steeper and stronger relationship between height and shoe size for pre-schoolers because they are growing quickly.

We are going to use the `lme4` package to fit mixed-effects models using random slopes and intercepts. This package has two main functions: the `lmer()` function fits linear mixed-effects models, and the `glmer()` function fits generalized linear mixed-effects models. We won't explore GLMMs in this tutorial, but you can fit them by building on the `lmer()` function with the `family` argument just as you would with a generalized linear model. We are going to specify our mixed-effects by adding them on to the formula we would normally include in our models:

- *formula + (1 | group)* specifies a random intercept model. The "group" is the name of the grouping variable from your data.
- *formula + (1 + predictor variable | group)* specifies a random slope and intercept model. The predictor variable(s) is the predictor variable from your formula over which you expect the group slopes to vary.

All the other formatting details for fitting `lmer()` models are the same as `lm()` models.

Note that we can also specify other random effects structures including random slope models without random intercepts, random slope and intercept models that force the slope and intercept to be uncorrelated, and nested random effects. If you are interested, this GitHub book provides a good overview of the options. Most design structures are well suited to the options I showed you above, so we won't go any farther here.

### 1c: Fitting mixed-effects models

We are going to fit some models to test the effects of temperature on response time using our response time data.

### Mini Challenge 1

Using `lmer()` and `lm()`, fit four models:

1. A mixed-effects model testing the effect of temperature on response time with random intercepts for the in the `animal` groups (call this model **mmod_intercepts**)
2. A mixed-effects model testing the effect of temperature on response time with random intercepts *and slopes* for the in the `animal` groups (call this model **mmod_slopes_intercepts**)
3. A linear model testing the effects of temperature on response time and including `animal` as a predictor, *without any random effects* (call this model **mod_lm_complex**)

4. A linear model testing the effect of temperature on response time *without any random effects* (call this model **mod_lm_simple**)

#————————————————————-

### 1c: Interpreting mixed-effects models

Let's take a look at our model summaries.

```
summary(mmod_intercepts)
summary(mmod_slopes_intercepts)
summary(mod_lm_simple)
summary(mod_lm_complex)
```

The first thing you'll likely notice is that while all models suggest there is a significantly negative effect of temperature on response time, the effect is much stronger in the mixed-effects models and the **mod_lm_complex** models. We will return to this finding later. For now, we need to examine the model summaries in more detail.

**Mixed-effects variance** At the top of your model summary, you'll see the heading "Random effects". This section of our model output provides the residual, intercept, (and slope variances in the case of our intercepts + slopes model). These numbers provide a lot of information about the importance of our random effects.

- The *intercept variance* tells us how much our groups differ in terms of their baselines or intercepts.
- The *temp (slopes) variance* tells us how much our groups differ in terms of the effect of temperature on response times.
- The *residual variance* tells us how much variance remains unexplained by either the random or fixed effects. If this value is large relative to the random effects variance, it suggests most of the variation in our response is *within* the groups rather than between them. Conversely, a relatively large random effects variance tells us most of the variation exists *between* groups.

Remember, in mixed-effects models we are interested in the variation among groups, so a larger number in the intercept and slope variances relative to the residual variance indicates that our random effects are doing some work.

### Mini Challenge 2

Nothing to code here—just think about this!

Your **mmod_intercepts** and **mmod_slopes_intercepts** models include different random effects structures. Comparing the residual variances between these two models, which of the structures do you think explains more variation in your groups? Do you think most of the variation occurs within or between groups?

#————————————————————-

**Residual degrees of freedom** The final thing we'll look at is the residual degrees of freedom. This information can't be easily found in your model summary, but the function `df.residuals()` provides it for any model object in R, taking the model object as its argument. Let's compare **mmod_intercepts** to **mod_lm_complex**.

```
df.residual(mod_lm_complex)
df.residual(mmod_intercepts)
```

Both of these models essentially specify a different baseline response time for each animal group. However, the random effects model (*mmod_intercepts*) does so with random intercepts. The degrees of freedom are much higher for this model; in fact, we only need one degree of freedom to account for the group-specific intercepts. As you know, the more degrees of freedom we have available, the more power we have to detect an effect. As we'll see shortly, this is because random effects models share information across groups, allowing them to precisely estimate effects we're interested in without eating up the many degrees of freedom we need to include the group variable as a fixed effect.

### 1d: Assessing mixed-effects model performance

It's generally good to use mixed-effects models if your data are structured in groups. But how do we know if these models are an improvement?

**AIC** We need to assess the performance or goodness-of-fit of our models. A simple way to do this is using AIC.

```
AIC(mmod_intercepts)
AIC(mmod_slopes_intercepts)
AIC(mod_lm_simple)
```

Our mixed-effects models are less than half of the AIC of the simple linear model! Our random effects must be explaining a lot of variation in the data.

**Marginal and conditional R-squared** We saw how we can use adjusted R-squared to assess the fit of our simple linear models. In linear mixed-effects models, we have two values to estimate:

- **Marginal R-squared**: Describes the proportion of variance explained by only the fixed effects
- **Conditional R-squared**: Describes the proportion of variance explained by *both* the random and fixed effects.

The function `r2()` from the `performance` package estimates marginal and conditional R-squared. It takes a model object as its argument.

### Mini Challenge 3

Use the `r2()` function to get the marginal and conditional R-squared for the **mmod_slopes_intercepts** model.

Based on the results, how important do you think the random effects are for explaining the variance in response times relative to the fixed effects?

#----------

## Part 2: Visualizing random effects

We just discussed how our random effects models use "partial pooling" do that the different groups in our data can exchange information to estimate the overall conditional effect. This is superior to running separate regressions for each group, where the groups would have no way to exchange information. It is also superior to not accounting for the group structure at all, where we would be oblivious to the variation that contributes to our effect.

Going back to our model outputs, you might remember that the effect sizes differed between our mixed-effects models and our simple linear regression. Let's look at the summaries for **mod_lm_simple** and **mmod_slopes_intercepts** again.

```
summary(mod_lm_simple)
summary(mmod_slopes_intercepts)
```

The mixed effects model with random slopes and intercepts clearly thinks there is a much stronger negative relationship between temperature and response time than the simple linear model. We will look at some plots to see why and to get a sense of the partial pooling effect.

### 2a: Models with complete pooling and without pooling

To get a sense of what our model sees when we don't pool information across groups, let's make a ggplot.

#### Mini Challenge 4

1. Use the `response_times` data to make a ggplot with `temp` on the x-axis and `response_time` on the y-axis.
2. Include a `geom_smooth()` coloured by `animal`.
3. Layer on another `geom_smooth()` that is not coloured by group. Colour this line black instead.

For both `geom_smooth()`, specify `se = F`, `method = "lm"`, and `linewidth = 0.5`.

#———————————————————-

This plot shows us what our relationships look like when we do not pool data across groups. The central black line is our slope and intercept from the "complete pooling" model, where the model function assumes that there is no structure in our data. In other words, the model is naive to the fact that we have multiple observations across multiple mice.

The coloured lines represent the slopes and intercepts for each individual mouse. However, since we've essentially fit a different regression for each mouse, the mice have no way to pool data.

### 2b: Models with partial pooling

Now let's compare this to our model with partial pooling. First, we need to predict the marginal and conditional effects from our mixed-effects model with random intercepts and slopes.

We saw previously how we could use the `predict()` function to make predictions of the fixed effect responses from our models. We can also use this function to make predictions about our conditional and fixed effects. the argument `re.form` specifies whether we want to predict the conditional effects (random + fixed effects) or marginal effects (fixed effects only). By default, this argument is set to `re.form = NA`, which means marginal effects. `re.form = NULL` tells the function to predict our conditional effects. We're going to predict both without specifying a vector of the predictor variable to predict over. When we don't specify a predictor vector, `predict()` just uses the existing data in our data we passed to the model.

```
mmod_effects <- response_times %>%
    mutate(marg_effects = predict(mmod_slopes_intercepts, re.form = NA),
        cond_effects = predict(mmod_slopes_intercepts, re.form = NULL))
```

We've now added our marginal and conditional effects to our original data. Next, we'll plot the effects to observe the results of partial pooling in comparison to our previous plot.

```
ggplot(mmod_effects, aes(x = temp)) + geom_line(aes(y = cond_effects,
    colour = animal)) + geom_line(aes(y = marg_effects), colour = "black") +
    labs(x = "temperature", y = "response time")
```

We can see all of the individual mouse lines have shifted to a more obvious downward slope. This has worked especially well for the mice with less data; the lines are mostly pulling in the direction of the mice with the most data that contribute most. The marginal effects slope (black line) has also shifted to align with the stronger negative relationship between temperature and response time. The marginal effects are using the pooled information from the random slopes and intercepts to come up with a consensus for the overall effect.